

# COMICCRAFT

## DOCUMENT 12: PROJECT EXECUTABLE FILES

### Step 1: Submission Checklist

| S.NO | ITEM TO SUBMIT                                         | SUBMITTED (YES / NO / NA)    |
|------|--------------------------------------------------------|------------------------------|
| 1    | Complete source code (all files and folders)           | ■ Yes                        |
| 2    | README / Setup Guide (instructions to run the project) | ■ Yes                        |
| 3    | requirements.txt / dependency file                     | ■ Yes                        |
| 4    | Database schema / seed files (if applicable)           | N/A (File-based storage)     |
| 5    | Environment configuration file (.env.example)          | ■ Yes                        |
| 6    | Deployed application URL (if hosted)                   | Local: http://127.0.0.1:8000 |
| 7    | APK / Executable binary (if applicable)                | N/A (Web application)        |
| 8    | Dockerfile / Containerization config (if applicable)   | ■ Yes (Optional)             |
| 9    | Test files and test results                            | ■ Yes (tests/)               |
| 10   | Demo video or walkthrough (if required)                | ■ Optional (can be added)    |

### Step 2: File / Folder Structure

ComicCraft/

```
■■■■ app/
■   ■■■■ __init__.py           # Package initializer
■   ■■■■ main.py               # FastAPI app entry point, middleware
■   ■■■■ routes.py             # All HTTP route handlers
■   ■■■■ config.py             # Configuration, settings, env loading
■   ■■■■ models.py             # Pydantic data models/schemas
■   ■■■■ schemas.py            # Request/response validation schemas
■   ■■■■ errors.py             # Custom error handling utilities
■   ■■■■ json_utils.py         # JSON parsing and extraction utilities
■   ■■■■ gemini_client.py      # Gemini API client wrapper
■   ■■■■ gemini_flash.py       # Gemini Flash outline generation
■   ■■■■ gemini_pro.py         # Gemini Pro story/narration generation
■   ■■■■ image_generator.py    # Stable Diffusion / FLUX image generation
■   ■■■■ layout_builder.py     # Comic layout assembly
■   ■■■■ exporters.py          # FPDF2 PDF export
■   ■■■■ pipeline.py           # Main generation pipeline orchestrator
■■■■ templates/
■   ■■■■ index.html            # Input form page
■   ■■■■ comic_preview.html    # Comic preview page
■   ■■■■ export_success.html   # Export confirmation page
■■■■ static/
■   ■■■■ css/
■   ■ ■■■■ style.css           # Application styles (dark glassmorphism)
```

```

■   ■■■ js/
■   ■   ■■■ main.js           # Client-side interactivity
■   ■■■ panels/              # Generated panel images (temp)
■   ■■■ exports/             # Generated PDF files (temp)
■■■ tests/
■   ■■■ __init__.py
■   ■■■ test_core.py         # Core module unit tests
■   ■■■ test_routes.py       # API route integration tests
■■■ .env.example              # Environment configuration template
■■■ .gitignore                # Git ignore rules
■■■ requirements.txt          # Python dependencies
■■■ requirements-ai.txt       # AI-specific dependencies (optional)
■■■ Dockerfile                # Containerization (optional)
■■■ README.md                 # Complete project documentation
■■■ run.py                    # Application runner script

```

### Step 3: Deployment / Access Details

| FIELD                       | DETAILS                                                                                           |
|-----------------------------|---------------------------------------------------------------------------------------------------|
| Hosted / Deployed URL       | <a href="http://127.0.0.1:8000">http://127.0.0.1:8000</a> (Local development)                     |
| Login Credentials (Demo)    | Not required (no authentication implemented)                                                      |
| Platform / Hosting Provider | Local development (Uvicorn ASGI server)                                                           |
| Repository Link             | <a href="https://github.com/roherror404/ComicCraft">https://github.com/roherror404/ComicCraft</a> |
| Demo Video Link             | (Optional - can be added upon request)                                                            |

### Step 4: Run Instructions

#### Clone the repository:

```

git clone https://github.com/roherror404/ComicCraft.git
cd ComicCraft

```

#### Create and activate virtual environment:

```
python -m venv venv
```

##### # Windows:

```
venv\Scripts\activate
```

##### # macOS/Linux:

```
source venv/bin/activate
```

#### Install dependencies:

```
pip install -r requirements.txt
```

#### # For GPU-accelerated image generation:

```
pip install -r requirements-ai.txt
```

#### Configure environment:

```
cp .env.example .env
```

# Edit .env and add your API keys:  
# GEMINI\_API\_KEY=your\_gemini\_api\_key\_here

Run the application:

```
uvicorn app.main:app --reload
```

Or using the runner script:

```
python run.py
```

Open in browser:

```
http://127.0.0.1:8000
```

Access Swagger API documentation:

```
http://127.0.0.1:8000/docs
```

Step 5: Known Issues / Limitations

| S.NO | KNOWN ISSUE / LIMITATION                                                                     | WORKAROUND / STATUS                                                                                           |
|------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| 1    | Image generation is GPU-intensive and slow on CPU-only machines (15-25s per 5 images)        | Use mock fallback mode for UI testing. For production, deploy on cloud GPU or use API-based image service     |
| 2    | Characters may appear visually inconsistent across different panels (different poses/styles) | Use detailed character descriptions in prompts. Future: implement character-consistent models like IP-Adapter |
| 3    | First run of image generation downloads large Stable Diffusion model (~2GB+)                 | Ensure stable internet connection. Model is cached locally after first download - subsequent runs are faster  |